

Лабораторная работа №2

Изучение и освоение методов обработки и сегментации изображений

Обработка и распознавание изображений, 317 группа, ММП
ВМК МГУ

Сычев Владимир Александрович

Апрель 2026

1 Введение

Целью данной лабораторной работы является изучение и практическое освоение методов обработки и сегментации изображений на примере задачи распознавания фишек игрового набора «Тантрикс». В рамках работы требовалось разработать программу, которая получает на вход изображение в формате BMP, выделяет на нем фишки, анализирует цветные линии на каждой фишке и выдает описание состава изображения.

Игровая фишка представляет собой черный правильный шестиугольник, на котором находятся три цветные линии: желтая, синяя и красная. Каждая линия может быть короткой дугой большой кривизны, длинной дугой малой кривизны или прямым сегментом. Всего в наборе используется десять различных типов фишек.

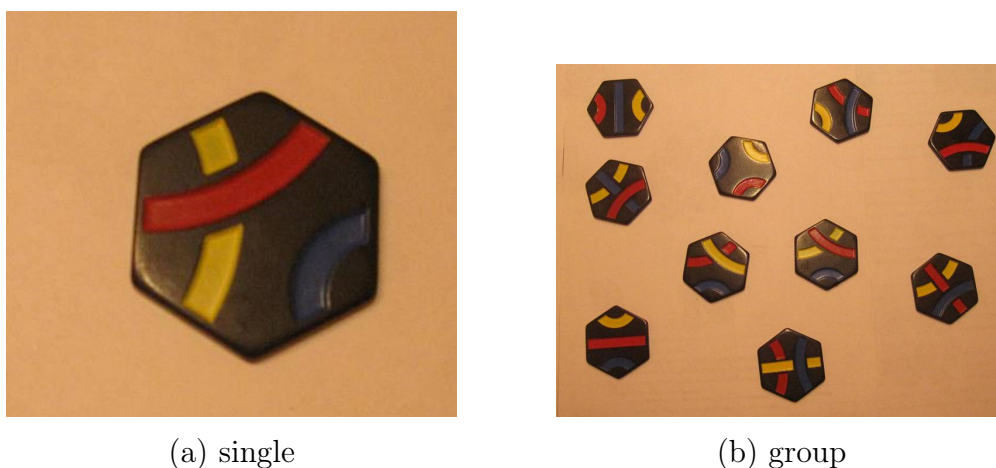
В работе была реализована программа, решающая задачи уровня Beginner и Intermediate. Она позволяет подсчитать количество фишек на групповом изображении, определить типы линий на одиночной фишке, найти номер одиночной фишки и определить номера всех фишек на изображении группы. Для решения использовались классические методы компьютерного зрения: цветовая сегментация в пространстве HSV, бинарные маски, морфологические операции, поиск контуров, анализ связных компонент и геометрический анализ формы шестиугольника.

2 Описание данных

В качестве входных данных используются изображения фишек «Тантрикс» в формате BMP. Изображения имеют различные размеры и могут содержать как одну фишку, так и несколько фишек в кадре. В простейших случаях на фотографии находится одна фишка, расположенная на светлом фоне, более сложные изображения содержат несколько фишек, которые могут иметь разные углы поворота.

Файлы типа `Single_*.bmp` используются для анализа одиночной фишки. На таких изображениях необходимо определить типы трех линий или номер фишки. Файлы типа `Group_*.bmp` содержат несколько фишек. Для них требуется либо подсчитать количество объектов, либо определить расположение и номер каждой фишки. В задании также присутствуют изображения типа `Path_*.bmp`, предназначенные для уровня Expert.

Пример входных объектов представлен на рисунке 1.



(a) single

(b) group

Рис. 1: Пример входных объектов.

3 Метод решения

Общая идея алгоритма состоит в том, чтобы сначала выделить черные шестиугольные области, соответствующие фишкам, а затем внутри каждой найденной фишки отдельно анализировать желтую, синюю и красную линии. Такой подход удобен для данной задачи, потому что фон на изображениях заметно светлее самих фишек, а цветные линии имеют достаточно выраженный цветовой тон.

На первом этапе изображение переводится из цветового пространства RGB в HSV. Для выделения самой фишки используется канал яркости V ,

так как корпус фишки темный, все пиксели с достаточно малым значением яркости считаются кандидатами на принадлежность объекту. В программе используется пороговая маска:

$$0 \leq H \leq 179, \quad 0 \leq S \leq 255, \quad 0 \leq V \leq 175.$$

После этого находятся внешние контуры темных областей. Контуры с малой площадью отбрасываются, а крупные области закрашиваются. Это позволяет получить сплошную маску фишки и убрать внутренние разрывы, которые появляются из-за цветных линий и бликов.

После выделения фишки начинается анализ цветных линий. Для каждой одиночной фишки строятся три отдельные маски: желтая, синяя и красная. Изображение фишки также переводится в HSV, после чего для каждого цвета выбирается свой диапазон. Для желтого цвета используется диапазон $H \in [16, 24]$, $S \in [175, 255]$, $V \in [100, 255]$. Для синего цвета используется диапазон $H \in [100, 178]$. Для красного цвета используется диапазон $H \in [0, 8]$, $S \in [100, 255]$, $V \in [100, 255]$. Полученные цветовые маски пересекаются с маской фишки, чтобы исключить пиксели вне объекта.

Цветные маски получаются шумными, поэтому они дополнительно очищаются морфологическими операциями. Сначала применяется открытие, которое удаляет небольшие шумы, затем применяется закрытие, которое соединяет небольшие разрывы внутри цветных линий. Слишком маленькие компоненты удаляются.

Пример выделения масок и границ представлен на рисунке 2.

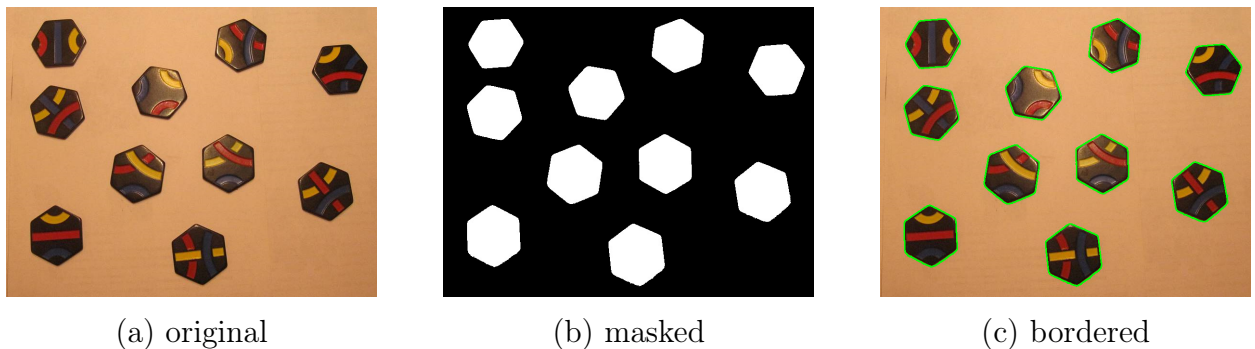


Рис. 2: Пример выделения масок и границ у фигур.

Для определения типа линий было принято решение анализировать, какие стороны шестиугольника соединяет каждая из них. Короткая линия соединяет смежные стороны, прямая линия соединяет стороны, лежащие друг напротив друга, а длинная линия соединяет оставшуюся пару сторон.

Алгоритм следующий. Для каждой фишки вычисляются центр через моменты бинарной маски:

$$x_c = \frac{m_{10}}{m_{00}}, \quad y_c = \frac{m_{01}}{m_{00}}.$$

После этого на контуре фишки ищется точка, наиболее удаленная от центра. Это есть первая вершина шестиугольника. Остальные вершины строятся поворотом радиус-вектора на углы, кратные 60° . Для этого используется матрицу поворота:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \begin{pmatrix} x - x_c \\ y - y_c \end{pmatrix}.$$

Так как на фотографии фишки могут быть немного искажены перспективой, найденные вершины дополнительно уточняются. В окрестности каждой примерной вершины выбирается реальная точка контура, которая находится дальше всего от центра. После этого центры сторон вычисляются как середины соседних вершин:

$$p_i = \frac{v_i + v_{i+1}}{2}.$$

Далее каждая цветная линия сравнивается с шестью центрами сторон. Для этого берутся все пиксели соответствующей цветовой маски, и для каждого центра стороны вычисляется минимальное расстояние до линии. Две стороны с наименьшими расстояниями считаются сторонами, которые соединяет данная линия. Если номера этих сторон равны s_1 и s_2 , то вычисляется циклическая разность:

$$d = \min(|s_1 - s_2|, 6 - |s_1 - s_2|).$$

Если $d = 1$, линия считается короткой, если $d = 2$, длинной, если $d = 3$, прямой.

Пример выделения цветных масок и нахождения центров сторон представлен на рисунке 3.

Для определения номера фишки используется словарь признаков для 10 типов фишек, представленных на рисунке 4.

Для каждой фишки хранится набор из шести значений:

$$[T_y, T_b, T_r, C_y, C_b, C_r],$$

где T_i — тип линии, а C_i — количество компонент связности для одной линии. Такой набор позволяет однозначно определить номер фишки. Например, для фишки из рисунка 1 запись имеет вид:

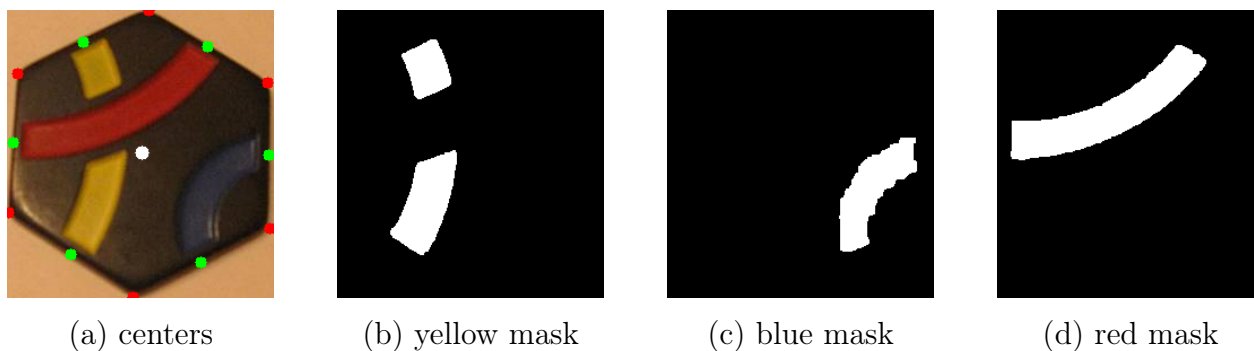


Рис. 3: Пример выделения цветных масок и нахождения центров сторон.



Рис. 4: Все типы фишек.

"7": ["long_arc", "short_arc", "long_arc", 2, 1, 1].

Для новой фишки вычисляется такой же вектор признаков и сравнивается со словарем. Если найдено точное совпадение, возвращается соответствующий номер.

4 Описание программной реализации

Программа реализована на языке Python. Для обработки изображений используются библиотеки OpenCV и NumPy. Графический интерфейс реализован с помощью CustomTkinter.

Основная часть алгоритма состоит из нескольких функций. Функция `read()` загружает изображение с диска. Функция `dark()` строит первичную маску темных областей, соответствующих корпусам фишек. Для групповых изображений используется функция `group_mask()`, в которой выполняется дополнительная обработка связанных компонент, эрозия, дилатация и проверка выпуклости объектов. Функция `count()` подсчитывает количество найденных фишек на групповой маске.

Для работы с одиночной фишкой используется функция `one_tile()`, которая находит самую крупную фишку и вырезает ее из изображения. Цветовые маски строятся функцией `color_mask()`. Геометрические точки фиш-

ки, то есть центр, вершины и центры сторон, вычисляются функцией `points()`. Типы цветных линий определяются функцией `line_types()`, а количество связанных компонент цветных масок — функцией `parts()`. Номер фишки определяется в функции `tile_num()`, после чего функция `rec()` объединяет все этапы распознавания одной фишки.

Интерфейс программы позволяет выбрать изображение, выбрать одно из четырех реализованных заданий и запустить обработку. Для заданий 1 и 4 программа принимает файлы вида `Group_*.bmp`, а для заданий 2 и 3 — файлы вида `Single_*.bmp`. В интерфейсе также добавлена кнопка справочника фишек.

5 Эксперименты

Для проверки работоспособности программы использовались предоставленные изображения одиночных фишек и групп фишек.

В первой задаче программа для группы фишек подсчитывала их количество. На итоговом изображении найденные фишки дополнительно выделялись контуром.

Во второй задаче устанавливалась корректность определения типов линий на одиночной фишке. Программа выводила результат в виде текстового описания, например: «короткая желтая», «длинная синяя», «длинная красная».

В третьей задаче проверялось, как программа определяет номер одиночной фишки.

В четвертой задаче проверялось, как программа определяет номера фишек на групповом изображении.

Программа корректно отработала на всех предоставленных изображениях фишек. Примеры представлены на изображениях 5, 6, 7, 8.

6 Выводы

В ходе выполнения лабораторной работы была разработана программа для обработки изображений фишек игрового набора «Тантрикс». Были реализованы основные этапы решения: выделение фишек на изображении, построение цветовых масок линий, определение геометрии шестиугольника, классификация типов линий и распознавание номера фишки.

Реализованный подход основан на классических методах компьютер-

ного зрения и не требует обучения модели. Благодаря тому, что структура объектов заранее известна, удалось использовать признаки, связанные с расположением линий относительно сторон шестиугольника. Такой способ позволяет достаточно просто и интерпретируемо определять типы линий и номера фишек.

Полученные результаты показывают, что выбранный метод подходит для решения задач начального и промежуточного уровней при достаточно хорошей сегментации фишек. Основным ограничением остается чувствительность к освещению, бликам и сильному соприкосновению фишек.

Список литературы

- [1] Гонзалес Р., Вудс Р. Цифровая обработка изображений. М.: Техносфера, 2006.

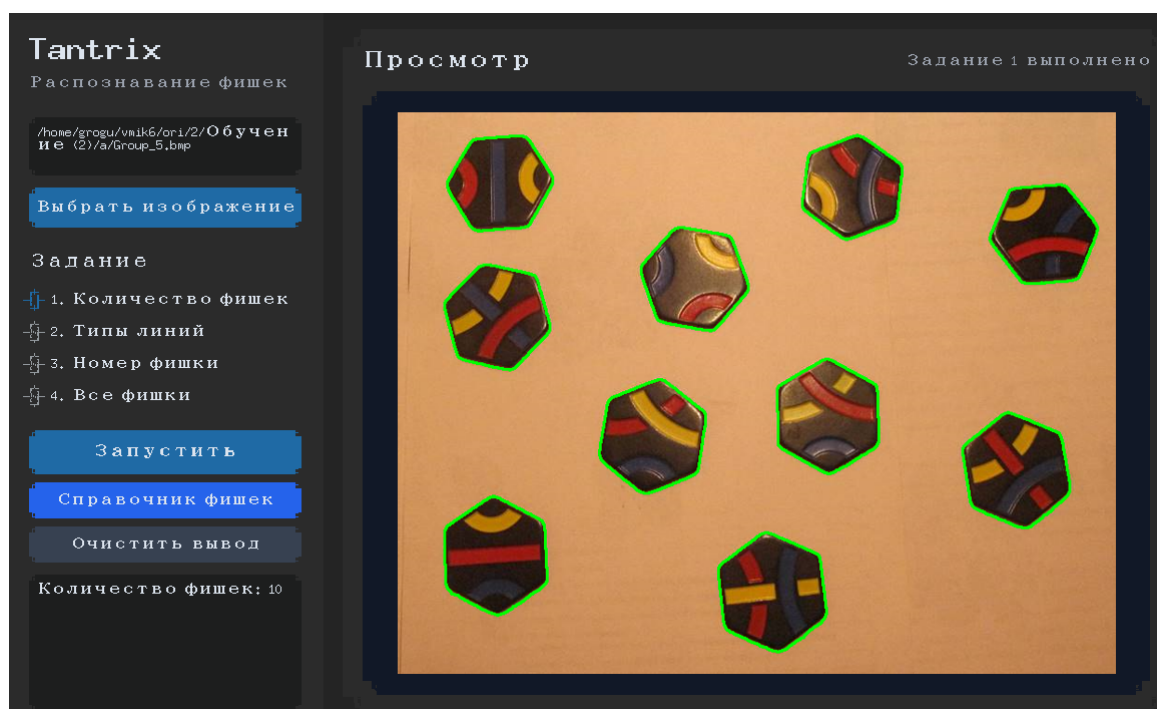


Рис. 5: Пример решения Задачи 1.

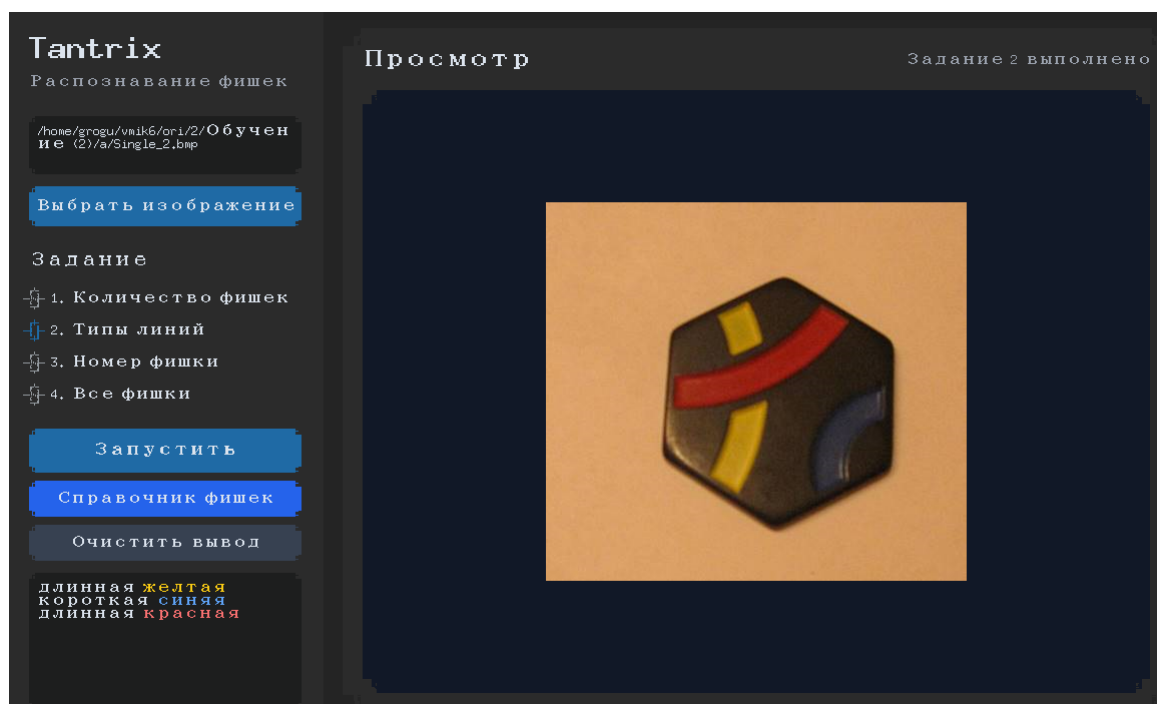


Рис. 6: Пример решения Задачи 2.

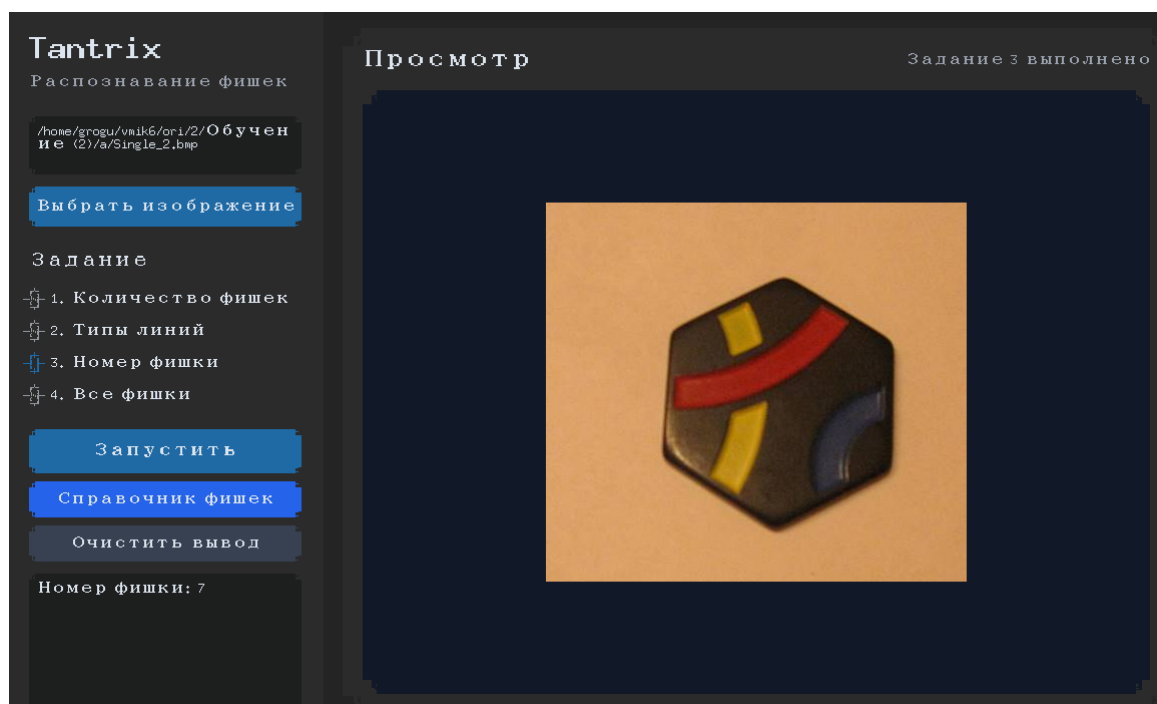


Рис. 7: Пример решения Задачи 3.

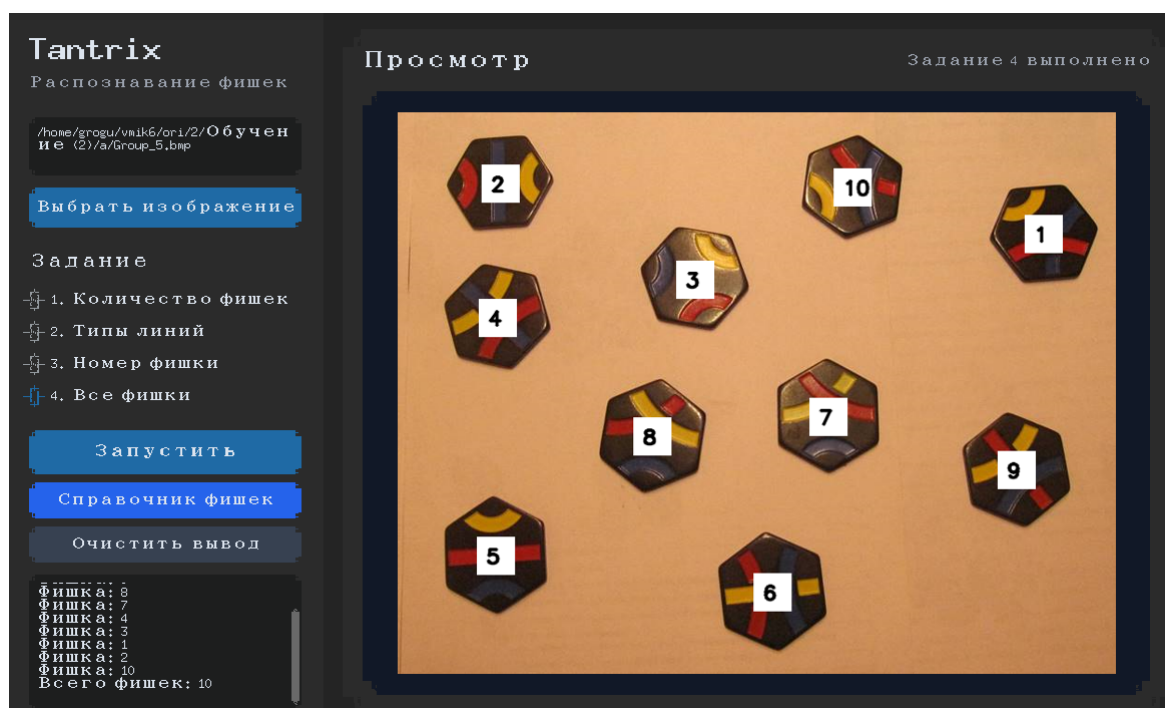


Рис. 8: Пример решения Задачи 4.